

flint front ends

Three ways to put an S3 bucket in front of a workload — **flint-lite**, **flint-lean** and **flint-passthrough** — with the multi-cluster and security consequences of each, and the one durable store they all share.

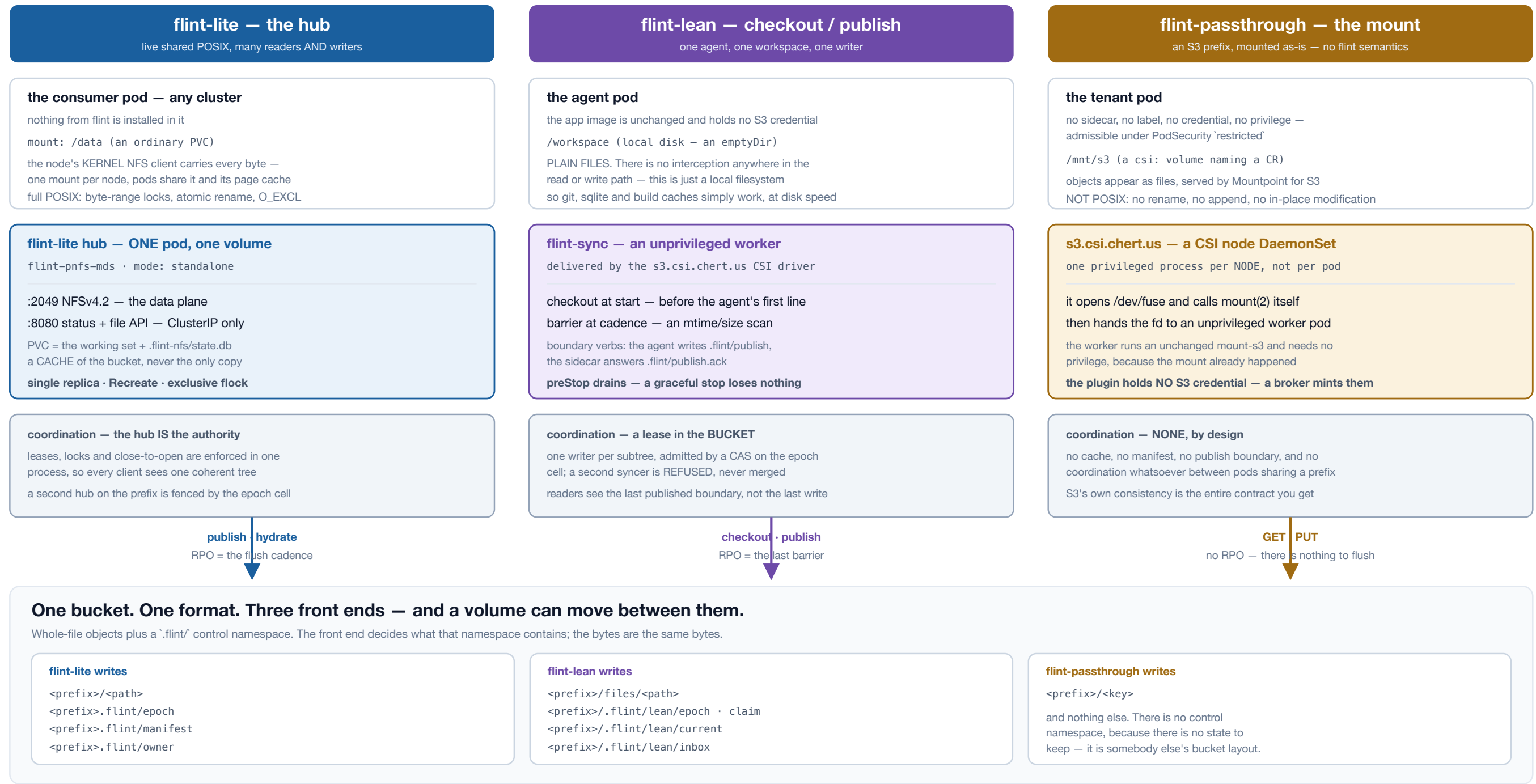
Scope. The architecture of the three front ends as built, the fleet shape each implies when one user launches agents across several Kubernetes clusters, what identity is actually enforced on the data path, and where S3 sits in all three. Every claim here is drawn from the repository at the revision noted below; where a guarantee has a stated ceiling, the ceiling is stated rather than rounded up.

Sources of record. `docs/plans/csi-node-mount-design.md` (the CSI delivery), `docs/plans/flint-lean-plan.md`, `docs/flint-lite-architecture.html`, `docs/flint-lean-architecture.html`, the three charts, and the code under `spdk-csi-driver/src/{s3csi,passthrough,tier,light_operator,lean_operator}` and `lean/sidecar/src`.

Regenerating. `docs/architecture/build.sh`. The diagrams are standalone SVG files under `diagrams/`, referenced rather than inlined, so they can be reused in any other format; the build also emits a Markdown rendition for conversion.

- 1 · Three front ends, one bucket** — the shape of the choice, and what is common underneath it.
- 2 · flint-lite — the hub** — one pod as the coherence authority, a PVC as cache, S3 as the copy.
- 3 · flint-lite across clusters** — one hub, many clusters, and the three things the fleet must supply.
- 4 · flint-lean — checkout and publish** — plain local files, one writer, a boundary the agent declares.

- 5 · flint-passthrough — the CSI mount** — a prefix mounted as-is, and where the privilege went.
- 6 · Where a user's token can and cannot go** — what each front end actually authenticates.
- 7 · Fleet shape and blast radius** — what you install per cluster, and what one compromised pod reaches.
- 8 · S3 — the durable store** — the live view, the published view, and the recovery point of each.
- 9 · Choosing** — ten dimensions, three columns, one decision.

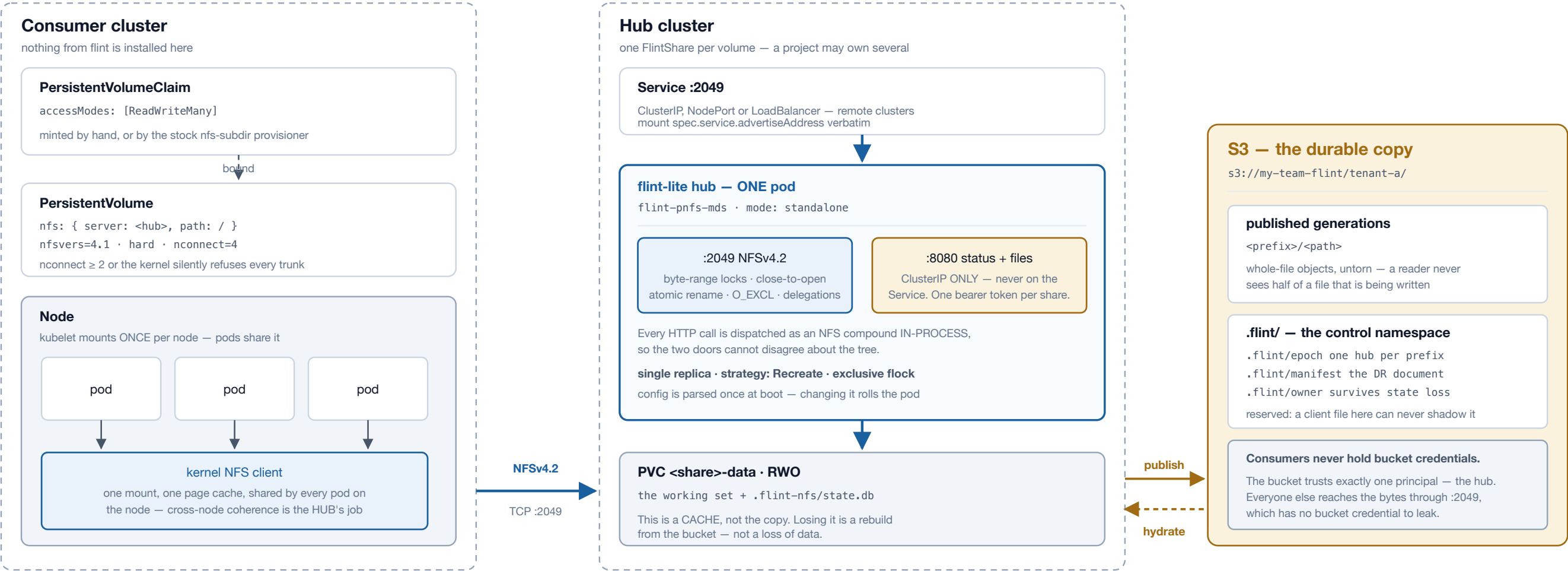


Read the columns as an escalation ladder, cheapest first. Passthrough asks nothing of the workload and gives it nothing back; lean gives real POSIX at local speed for one writer; the hub is the only one of the three that can make several pods share one live tree.

Read the columns as an escalation ladder, cheapest first. Passthrough asks nothing of the workload — nine lines of pod spec — and gives it nothing back beyond the objects: no POSIX, no coordination, no boundary. **Lean** gives a real local filesystem at disk speed to one writer, and a durable point that writer can name. **The hub** is the only one of the three where several pods, in several clusters, can share one live tree with locks that mean something.

The common floor is the bucket format, and it is what makes the choice reversible. All three write whole-file objects; what differs is the `.flint/` control namespace beside them, which exists to answer questions the object listing cannot — who holds the write lease, what the last coherent boundary contained, what the whole tree is for a restore. Passthrough writes no control namespace at all, which is exactly why it can be pointed at a prefix somebody else's tooling already owns. Because the formats are common and mutually convertible, a volume that outgrows one front end can move to another rather than being re-plumbed.

One pod serves real POSIX over NFSv4.2 to any number of pods in any number of clusters. The PVC beside it is a working set; an S3 prefix is the durable copy. A second listener exposes the same filesystem over HTTP for callers that cannot mount. One hub serves exactly one prefix — a project that needs several volumes runs several hubs.



What the hub is for, and what it costs

It is the coherence authority

One process holds every lease, lock and open, so N pods across N clusters see ONE tree. This is the only front end of the three where two writers can share a directory live.

That authority is also the constraint: one pod, one replica, Recreate. A second hub on the prefix would be split-brain, which the epoch cell fences and the operator refuses up front.

One hub serves one prefix. Several volumes ⇒ several hubs.

S3 is the durability, and the RPO is the cadence

The PVC is a working set the hub flushes to the bucket on a timer. Everything durable is in S3; everything fast is on disk.

A cold read hydrates from the bucket first, and both doors say so rather than blocking forever: NFS parks the caller with NFS4ERR_DELAY, HTTP answers 503 + Retry-After.

Losing the PVC is a rebuild. Losing the bucket is a loss.

Only one of those two is backed up, and it is not the disk.

The two doors are secured differently

:2049 is AUTH_SYS. The client asserts its own uid and the server takes it, so POSIX modes are evaluated against a SELF-DECLARED identity. Reachability is the real boundary.

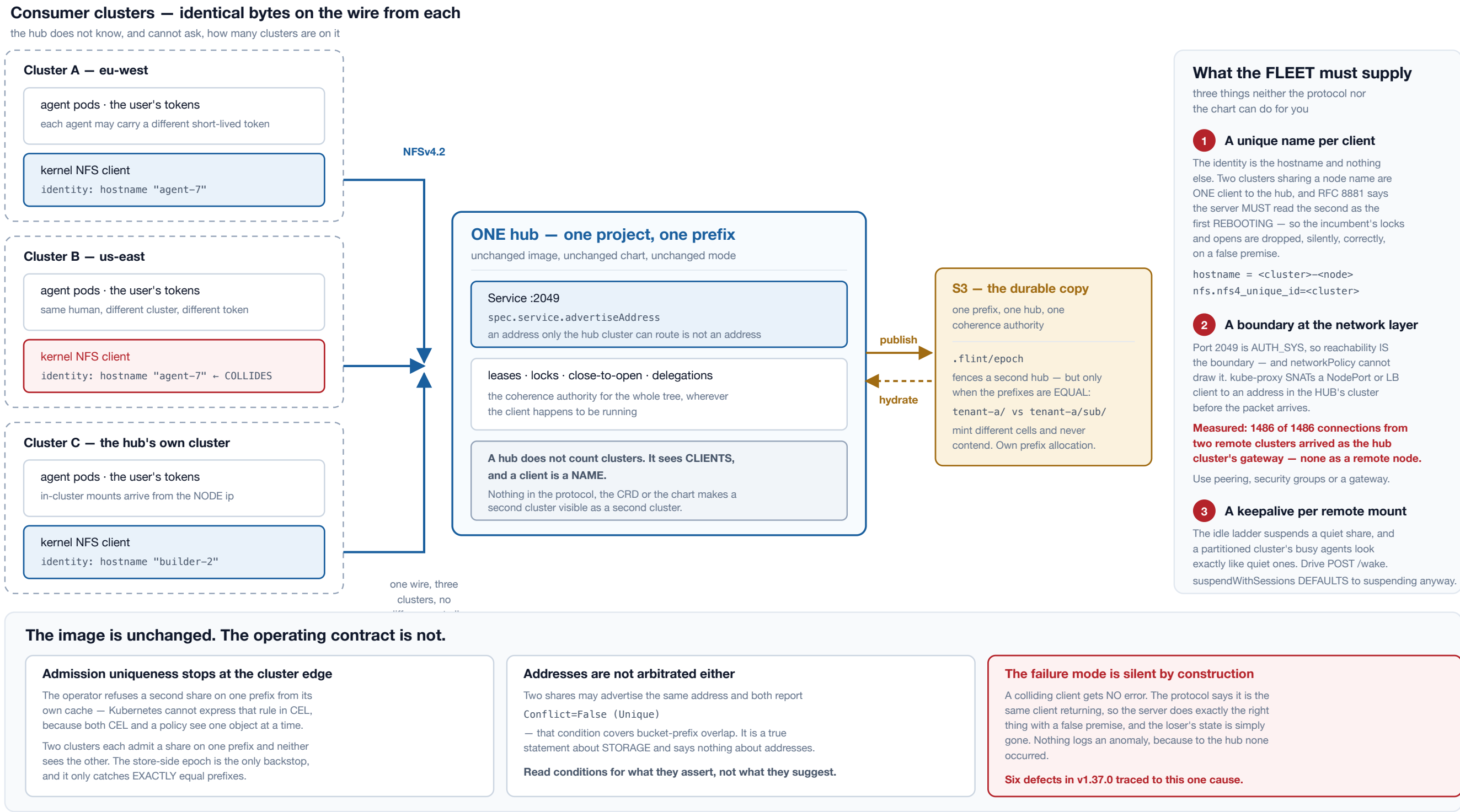
:8080 can rewrite every file in the share, so it stays ClusterIP-only, never joins the Service, and mounts no routes at all without a bearer token — 404, not 401.

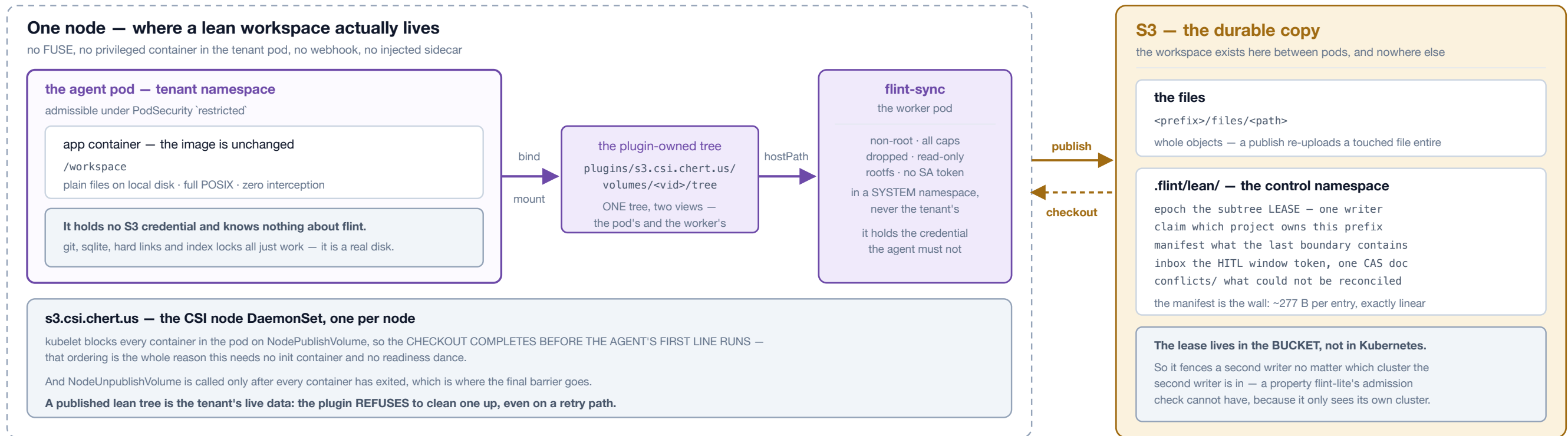
That token is per-SHARE, not per-user. See the identity page.

The hub's value and its constraint are the same fact. One process holds every lease, lock and open, so N pods across N clusters see one coherent tree — and that is why it is one pod, one replica, **Recreate**, behind an exclusive flock. Two hubs on one prefix would be split-brain, which the store-side epoch cell fences and the operator refuses up front. Consumers install nothing: the data path is the node's own kernel NFS client, one mount per node, shared by every pod on it along with its page cache.

The disk is a cache; the bucket is the copy. The hub flushes the working set to S3 on a cadence, so the recovery point is that cadence and losing the PVC is a rebuild rather than a loss. A cold read hydrates on demand and both doors say so rather than hanging — NFS parks the caller with NFS4ERR_DELAY, HTTP answers 503 + Retry-After. **The two doors are secured differently, and only one of them has a credential.** Port 2049 is AUTH_SYS, so the client asserts its own uid and reachability is the real boundary; port 8080 can rewrite every file in the share, so it stays ClusterIP-only, never joins the Service, and mounts no routes at all without a bearer token — 404, not 401. That token is per-share, not per-user, which page 6 takes up.

One hub can serve agents in several clusters at once. The wire is ordinary NFSv4.2 and the hub does not know, or care, how many clusters are on it — which is precisely the problem. Three things the protocol and the chart cannot supply become the fleet's responsibility, and every one of them fails silently if it is missed.





The loop – one agent, start to durable

- 1 pod starts**
checkout runs first — the agent never sees an empty tree
 - 2 the agent works**
local files at local speed; the syncer publishes at cadence anyway
 - 3 the agent declares a boundary**
`echo > .flint/publish`
one file write — the entire API surface
 - 4 the sidecar answers**
`.flint/publish.ack`
ok ⇒ the bytes are in S3 · partial · refused-fenced

Gated mode — durability and visibility become separate events

the upload lane — always running

every changed file becomes a NEW OBJECT VERSION as soon as it is scanned. Durable immediately.

And INVISIBLE to readers.

A crash in this lane loses nothing, because nothing here was ever cited.

the citation — ONE CAS

installs the whole pending set at once, naming version ids. There is no copy phase, so no half-boundary exists.

A reader sees the whole change or none of it.

Gated is REFUSED without a lag bound — unbounded staleness is impossible by construction, not convention.

Three questions, in order – the cheapest disqualifier first

1 · Does the workspace fit the disk?

The whole tree is materialized. There are no stubs and no eviction, so this is the hard gate — and it is about the FILE COUNT before the byte count: the v1 cap is ~250k files, and it falls out of the manifest size, not the bytes.

No ⇒ this is not the front end you want.

2 · Is one writer per subtree enough?

The lease admits exactly one syncer. A second is REFUSED, not merged — which is the honest behaviour, but it does mean two agents cannot share one live tree here. Several agents integrate through a git host instead.

No ⇒ the hub, for live shared POSIX

3 · Is snapshot freshness acceptable?

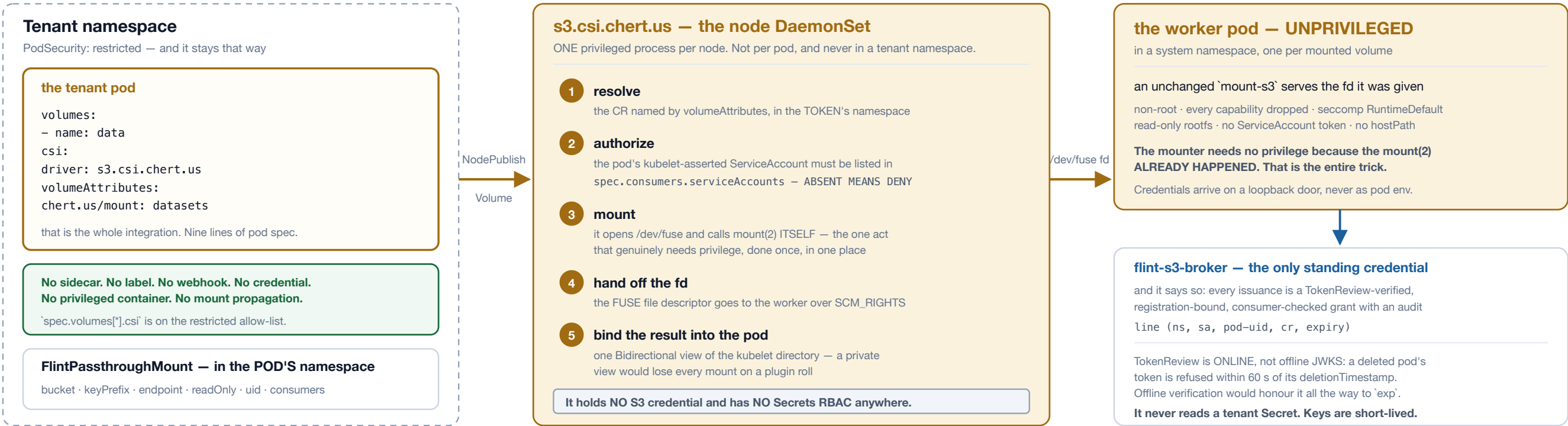
Readers see the last published boundary, never your last write. A hard kill loses at most floorSecs (default 60 s); a graceful stop loses nothing, because preStop drains. Gated narrows WHAT a reader sees, never HOW STALE it is.

No ⇒ the hub. Yes ⇒ lean is the cheapest thing that w

The ordering is the architecture. kubelet blocks every container in the pod on `NodePublishVolume`, so the checkout completes before the agent's first line runs — no init container, no readiness dance — and calls `NodeUnpublishVolume` only after every container has exited, which is where the final barrier goes. In between, the app touches plain files on real local disk with zero interception, so git, sqlite, hard links and index locks simply work. The app container holds no S3 credential; the unprivileged worker does, in a system namespace, behind a loopback door the app cannot reach.

The boundary verbs are files, because the workspace is the interface. An agent that can write a file can declare a coherent point — `echo > .flint/publish` — and learn when its bytes are durable, from `.flint/publish.ack:ok` means the bytes are in S3, **refused–fenced** means this syncer lost the lease and is saying so rather than leaving the agent waiting forever. No client library, no credential, no network path. **Gated mode answers a different question from cadence:** cadence asks how stale a reader may be, gated asks whether a reader may see half a logical change — and answers no, by uploading every changed file as a new version immediately (durable, and invisible) and installing the whole pending set with one CAS. Gated is refused without a lag bound, so unbounded staleness is impossible by construction rather than by convention.

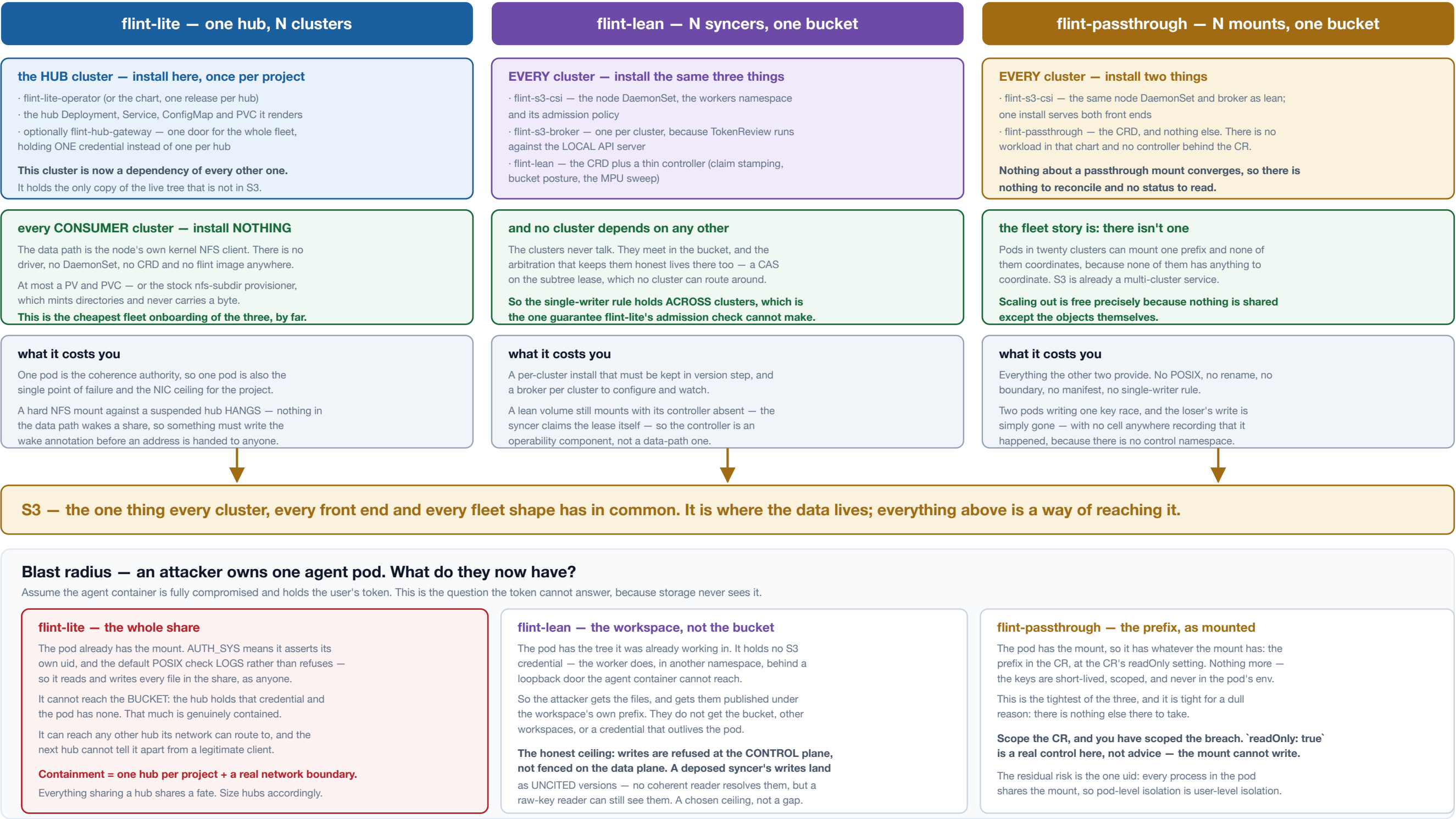
An S3 prefix, mounted into a pod as-is by Mountpoint for S3, with no flint semantics layered over it. There is no controller and no status, because nothing about a passthrough mount converges. What there is, is a careful answer to the question of who holds privilege — and the answer is: not the tenant.



The answer, for all three: it does not reach the data path.
Every front end authenticates the WORKLOAD, never the human behind it.



Two practical questions for anyone running agents on more than one cluster: what do I have to install in each of them, and if one agent pod is fully compromised, what does the attacker now have? The three front ends answer both differently, and the cheapest to install is not the one with the smallest blast radius.



Underneath all three front ends there is one durable store, and it is S3. Everything above it — the hub's disk, the lean workspace, the FUSE mount — is a way of reaching bytes whose home is the bucket. Two views of those bytes exist, with different contracts, and confusing them is the most common way to get hurt here.

The LIVE view — strong, and never in the bucket

a hub's port 2049 and its file API, or a lean workspace's local disk

close-to-open · byte-range locks · atomic rename · no RPO lag

Reads and writes land on the working set, through whatever holds coherence for it — the hub process, or the agent's own kernel.

Who reads it: front doors, editors, agents — anything touching a tree that is being changed right now.

The hub's live view offers one thing no real S3 can: assembly under a temp name, completed by an ATOMIC RENAME under a deny-both OPEN — an atomic, exclusive multipart-complete.

flint-lean has no live view to share at all: its live tree is local disk, visible to exactly one pod.

The PUBLISHED view — the bucket, and the durable copy

RPO-consistent snapshots per subtree, made of untorn whole-file generations

whole objects · versioned · read-only against a live subtree

A reader never sees half a file. It may well see a file from before your last write — that is the contract, not a defect.

Who reads it: project browsers, DR, analytics, and any non-mount access — plain S3, with no flint server in the read path at all.

Do not write into a live subtree from the console.

A hand-made PUT under a prefix a hub or syncer owns has three bad fates: overwritten at the next flush, reported as a conflict, or silently uncited. The bucket is the durable copy, not a second door.

Inside the prefix — the bytes, and the cells that make concurrency safe

Every cell below exists to answer one question that cannot be answered from the object listing alone.

flint-lite writes

<prefix>/<path>
the published generations

.flint/epoch
"who is the hub right now?" — a lease, so a second hub is fenced instead of interleaved

.flint/manifest
"what is the whole tree?" — DR from one GET

.flint/owner

flint-lean writes

<prefix>/files/<path>
the workspace, file for file

.flint/lean/epoch · claim
"who may write this subtree?" — one holder, enforced by CAS, across every cluster

.flint/lean/current
"what does the last boundary contain?"

.flint/lean/inbox · conflicts/

flint-passthrough writes

<prefix>/<key>
the objects, and nothing else

There is no control namespace here, and that is the entire feature.

A prefix owned by somebody else's tooling stays exactly as it was. flint adds no cells, claims no lease and leaves no trace when the pod goes away.

You also get no answer to any of the questions on the left.

Reserved, and enforced

`flint` is control state. A client file under it can never shadow a control object, and a NESTED nested/.flint/epoch is another share's namespace — never tiered into this one. Both are tested, because both were once a way to overwrite a live share's lease.

The formats are mutually convertible: a volume can move between front ends, and DR import is the same verb in each. Choosing today does not strand data.

What "durable" actually means here — the recovery point, per front end

flint-lite — RPO is the flush cadence

The PVC is a working set the hub flushes on a timer. Losing the disk is a REBUILD from the bucket, not a loss.

A cold read hydrates on demand and both doors say so rather than hanging: NFS returns NFS4ERR_DELAY, HTTP returns 503.

Hibernation deletes the PVC, so it is VERIFIED, never assumed:
the operator scales back to one, polls until the hub reports rpoClean, and only then deletes. A share with no bucket reports rpoClean: null — which is a refusal, not a pass.

Waking a hibernated share re-imports the whole tree (~79 s).

flint-lean — RPO is the last barrier

A hard pod kill loses at most floorSecs of work (default 60 s). A graceful stop loses nothing: preStop runs a final barrier.

A container restart over a live tree RESUMES from its baseline — it does not re-check-out, and does not lose the delta.

Publish at a breakpoint and the RPO becomes zero, knowably:
echo > .flint/publish → .flint/publish.ack = ok
means the bytes are in S3. That is the difference between "probably durable" and "durable, and I was told so".

Gated mode adds versions; retention must outlast the crash window.

flint-passthrough — there is no RPO

Nothing is buffered on flint's behalf, so nothing can be lost by flint. A completed PUT is durable the instant S3 says so, and an interrupted one simply did not happen.

This is the strongest durability story of the three — and it is strongest because flint is not in it.

The trade is on the other side of the ledger: a write is a whole new object, there is no rename to make it atomic against a reader, and two writers racing one key resolve as S3 resolves them — last writer wins, with nothing recording that a race occurred, because there is no cell in which to record it.

The live view is strong and is never in the bucket. It is a hub's port 2049 and file API, or a lean workspace's local disk: close-to-open, byte-range locks, atomic rename, no RPO lag. The published view is the bucket: RPO-consistent snapshots per subtree, made of untorn whole-file generations. A reader of the bucket never sees half a file, and may well see a file from before your last write — that is the contract, not a defect. Anything that is not a mount reads the published view as plain S3, with no flint server in the read path at all. Do not write into a live subtree from the console: a hand-made PUT under a prefix a hub or syncer owns is overwritten at the next flush, reported as a conflict, or silently uncited.

Every control cell exists to answer a question the object listing cannot. Who is the hub right now, and may I become it — the epoch lease. What does the last coherent boundary contain — the manifest, which also makes disaster recovery a single GET rather than a crawl. Who owns this prefix, and did state loss change that — the claim and owner cells. The claim is checked on the data path itself: a syncer reads it before it takes the lease and refuses a prefix another project holds, so refuse-foreign does not wait on an operator's verdict. Passthrough writes none of them, which is why it can be pointed at a prefix another system already owns and leaves no trace when the pod goes away; it also gets no answer to any of those questions. .flint is reserved and enforced from both sides: a client file cannot shadow a control object, and a nested .flint/ is another share's namespace, never tiered into this one.

The recovery point differs, and it is worth stating in the language of a hard pod kill. Lite loses up to the flush cadence, and its PVC is a cache — losing the disk is a rebuild. Lean loses at most floorSecs (default 60 s), zero on a graceful stop, and zero-and-acknowledged if the agent declares a boundary; a stop whose drain could not finish keeps the tree on the node, under the plugin's undrained/, and says so on the pod. Passthrough has no recovery point at all, because nothing is buffered on flint's behalf: a completed PUT is durable and an interrupted one did not happen. That is the strongest durability story of the three, and it is strongest because flint is not in it.

Ten dimensions, three columns. Read it down a column rather than across a row: each front end is coherent with itself, and the rows that look like weaknesses are usually the price of a strength two rows up.

dimension	flint-lite	flint-lean	flint-passthrough
What the pod sees the shape of the thing	a shared NFS mount, served by one hub pod that every client reaches over the network — one tree, many pods, live	plain files on local disk, checked out before the first line runs and published back on a boundary — one pod's own tree	an S3 prefix presented as files by Mountpoint for S3 — objects, with a directory-shaped view over them
POSIX fidelity what actually works	Full, and shared. byte-range locks, atomic rename, O_EXCL — across pods and clusters	Full, and local. it is a real filesystem, so git, sqlite and hard links work — for one pod	None to speak of. no rename, no append, no in-place write — at any setting, deliberately
Concurrent writers per tree	Many, coordinated. this is the only front end where two pods can edit one directory and both be right	Exactly one, enforced. a second syncer is REFUSED, not merged — several agents integrate through git instead	Many, uncoordinated. two writers on one key race; the loser's write is gone and nothing records it
What a reader sees the consistency contract	the live tree, strongly — close-to-open, no RPO lag, through the coherence authority	the last PUBLISHED boundary, never your last write; gated mode makes it all-or-nothing	whatever S3 currently holds, with S3's own consistency and nothing added
Across clusters the fleet shape	ONE hub, mounted from N clusters. Works unchanged — but needs unique client names, a network boundary and a keepalive.	N independent syncers, one bucket. Clusters never talk; the lease lives in the bucket, so single-writer holds ACROSS clusters.	N independent mounts, one bucket. There is no fleet problem because there is nothing shared to get wrong.
Identity that is enforced the user's token reaches NONE of these	Nobody, on the data path. AUTH_SYS: the client asserts its own uid. The POSIX check defaults to LOG-ONLY.	The POD's ServiceAccount. kubelet-minted, pod-bound, verified ONLINE by TokenReview, checked against `consumers`.	The POD's ServiceAccount. Identical chain to lean — same broker, same registration binding, same audit line.
Isolating user A from B when both launch agents on several clusters	Give each project its own hub and prefix, and bound reachability outside Kubernetes. Everything sharing a hub shares a fate.	Give each user their own workspace prefix. Keys are scoped to it, the lease fences it, and `consumers` says who may mount it.	Give each user their own CR and pod. The mount presents ONE uid, so pod-level isolation IS user-level isolation.
Installed per cluster in a consumer cluster	Nothing. the node's own kernel NFS client is the data path; at most a PV and PVC	the s3.csi.chert.us CSI driver, a broker, and the lean CRD + thin controller — in EVERY cluster (the broker must be local: TokenReview is local)	the s3.csi.chert.us CSI driver, a broker, and the CRD — in every cluster. One CSI install serves lean and passthrough together.
Recovery point what a hard kill costs	the flush cadence. The PVC is a cache — losing it is a rebuild from S3, not a loss.	the last barrier: ≤ floorSecs (default 60 s), or zero and acknowledged if you declare one.	None — nothing is buffered. a completed PUT is durable; an interrupted one did not happen.
Choose it when the one-line answer	Several pods must share one LIVE tree. Shared workspaces, cross-pod locking, a dataset too big for a pod's disk, or anything that needs a real filesystem two pods can see.	One agent owns a workspace that fits on disk. The agent-harness shape: a repo-sized tree, full POSIX at local speed, durable at breakpoints, and agents that integrate through a git host.	You want the bucket, not a filesystem. Read-mostly datasets, model weights, artifacts — a prefix another system already owns, that you want visible in a pod without changing it.

The three are not tiers of the same product — they are three different bargains over one bucket.

Read the table down a column, not across a row: each front end is coherent with itself. Lite trades a single point of coordination for a tree several pods can share. Lean trades sharing for a real local filesystem and a boundary you can name. Passthrough trades everything for the smallest possible thing that works. And because the bucket format is common, a volume can change its mind later.

The short form. Choose **flint-lite** when several pods must share one live tree — shared workspaces, cross-pod locking, a dataset larger than a pod's disk, or anything needing a filesystem two pods can both see. Choose **flint-lean** when one agent owns a workspace that fits on disk and wants full POSIX at local speed with a durable point it can name; several agents then integrate through a git host rather than through the bucket. Choose **flint-passthrough** when you want the bucket rather than a filesystem — read-mostly datasets, weights, artifacts, or a prefix another system already owns that you want visible inside a pod without changing it.

The multi-cluster and security rows are the ones most often decided late and regretted. If one user's agents must be isolated from another's, none of the three will do it for you inside the data path: the unit of isolation is the prefix, and the thing that enforces it is either a network boundary around a hub, or a broker scoping short-lived keys to a CR. Decide the prefix layout first and the front end second. **And because the bucket format is common and the front ends are mutually convertible, a volume can change its mind later** — which is the strongest argument for starting with the cheapest bargain that works rather than the most capable one.